

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №4
по курсу «Программирование графических процессоров»

Работа с матрицами. Метод Гаусса.

Выполнил: *Д.Н. Путилин*

Группа: *8О-407Б*

Преподаватель: *А.Ю. Морозов*

Москва, 2024

Условие

1. Цель работы.

Использование объединения запросов к глобальной памяти. Реализация метода Гаусса с выбором главного элемента по столбцу. Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust. Использование двумерной сетки потоков.

Исследование производительности программы с помощью утилиты nvprof.

2. Вариант 3.

Необходимо решить систему уравнений $Ax = b$, где A - квадратная матрица $n \times n$, b - вектор-столбец свободных коэффициентов длиной n , x - вектор неизвестных.

Входные данные. На первой строке задано число n - размер матрицы. В

следующих n строках, записано по n вещественных чисел - элементы матрицы. Далее записываются n элементов вектора свободных коэффициентов. $n \leq 10^4$.

Выходные данные. Необходимо вывести n значений, являющиеся элементами вектора неизвестных x .

Программное и аппаратное обеспечение

Compute capability : 7.5

Name : Tesla T4

Total Global Memory : 15835660288

Shared memory per block : 49152

Registers per block: 65536

Warp size : 32

Max threads per block : (1024, 1024, 64)

Max block : (2147483647, 65535, 65535)

Total constant memory : 65536

Multiprocessors count : 40

Метод решения

Метод решения системы линейных уравнений можно описать следующим образом: сначала матрица преобразуется к треугольному виду с использованием метода Гаусса, после чего значения неизвестных вычисляются на процессоре за квадратичное время.

Процесс приведения матрицы к треугольному виду можно разделить на три ключевые операции: **поиск максимума**, **перестановка строк** и **обновление строк**, каждая из которых может быть реализована параллельно.

- 1. Поиск максимума.** Эта операция заключается в определении элемента с наибольшим модулем в заданном столбце. Используем для этого библиотеку Thrust. При этом также добавляем компаратор, чтобы брался максимальный элемент по модулю.
- 2. Перестановка строк.** Цель этой операции — исключить деление на ноль. Она заключается в обмене текущей строки с той, где находится элемент с максимальным модулем в текущем столбце.

3. **Обновление строк.** Для всех строк, находящихся ниже текущей, выполняется операция вычитания текущей строки, умноженной на соответствующий коэффициент. Это приводит к обнулению значений в текущем столбце для строк, расположенных ниже.

Описание программы

Программа реализована в одном файле и включает ядра `swap_kernel` и `update_kernel`, которые выполняют функции перестановки строк и обновления элементов соответственно. В ядре `update_kernel` отсутствует прямое обнуление элементов под диагональю, что позволяет избежать состояния гонки и исключить ненужные вычисления.

Для строки с индексом `pos` обновление значений остальных элементов осуществляется по следующим формулам (где i — индекс строки, а j — индекс столбца):

1. Вычисляется коэффициент:
$$k = -\text{matrix}[i + \text{pos} * n] / \text{matrix}[\text{pos} + \text{pos} * n]$$
2. Затем элементы матрицы обновляются по правилу:
$$\text{matrix}[i + j * n] += k * \text{matrix}[\text{pos} + j * n]$$

В функции `main` выполняются ввод данных, построение исходной матрицы и нахождение значений неизвестных методом обратного хода. Для использования функции поиска максимального значения (`max`) из библиотеки `Thrust` был определён компаратор, который сравнивает элементы по модулю, чтобы корректно обрабатывать их абсолютные значения.

Результаты

Во всех тестах для одного размера матрицы будет использована одна и та же случайная матрица без линейно зависимых строк и столбцов. Время в таблице приведено в миллисекундах.

	10^1	$3 \cdot 10^1$	10^2	$3 \cdot 10^2$	10^3	$3 \cdot 10^3$	10^4
<(16,16), (16,16)>	2.86	2.37	8.05	28.09	122.13	1043.39	41284.6
<(32,32), (32,32)>	2.88	2.66	7.91	25.26	109.43	877.68	19689.1
<(64,64), (64,64)>	2.84	2.27	7.50	23.01	78.64	276.71	1030.87
<(128,128), (128,128)>	2.61	2.30	7.55	22.57	78.66	266.53	916.90
CPU	3.51	3.35	21.91	283.67	2735.72	74589.1	-

Вызовы в профилировщике:

swap(double *, int, int, int) (1024, 1, 1)x(1024, 1, 1), Context 1, Stream 7, Device 0, CC 7.5

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
lltex__data_bank_conflicts_pipe_lsu_mem_shared.avg		0
lltex__data_bank_conflicts_pipe_lsu_mem_shared.max		0
lltex__data_bank_conflicts_pipe_lsu_mem_shared.min		0
lltex__data_bank_conflicts_pipe_lsu_mem_shared.sum		0
lltex__t_sectors_pipe_lsu_mem_global_op_st.avg	sector	5.05
lltex__t_sectors_pipe_lsu_mem_global_op_st.max	sector	202
lltex__t_sectors_pipe_lsu_mem_global_op_st.min	sector	0
lltex__t_sectors_pipe_lsu_mem_global_op_st.sum	sector	202
lltex__t_sectors_pipe_lsu_mem_local_op_ld.avg	sector	0
lltex__t_sectors_pipe_lsu_mem_local_op_ld.max	sector	0
lltex__t_sectors_pipe_lsu_mem_local_op_ld.min	sector	0
lltex__t_sectors_pipe_lsu_mem_local_op_ld.sum	sector	0
sm__sass_inst_executed_op_local.avg	inst	0
sm__sass_inst_executed_op_local.max	inst	0
sm__sass_inst_executed_op_local.min	inst	0
sm__sass_inst_executed_op_local.sum	inst	0
smsp__sass_branch_targets_threads_divergent.avg		0
smsp__sass_branch_targets_threads_divergent.max		0
smsp__sass_branch_targets_threads_divergent.min		0
smsp__sass_branch_targets_threads_divergent.sum		0

update(double *, int, int) (32, 32, 1)x(32, 32, 1), Context 1, Stream 7, Device 0, CC 7.5

Section: Command line profiler metrics

Metric Name	Metric Unit	Metric Value
lltex__data_bank_conflicts_pipe_lsu_mem_shared.avg		0
lltex__data_bank_conflicts_pipe_lsu_mem_shared.max		0
lltex__data_bank_conflicts_pipe_lsu_mem_shared.min		0
lltex__data_bank_conflicts_pipe_lsu_mem_shared.sum		0
lltex__t_sectors_pipe_lsu_mem_global_op_st.avg	sector	70

lltex__t_sectors_pipe_lsu_mem_global_op_st.max	sector	288
lltex__t_sectors_pipe_lsu_mem_global_op_st.min	sector	0
lltex__t_sectors_pipe_lsu_mem_global_op_st.sum	sector	2,800
lltex__t_sectors_pipe_lsu_mem_local_op_ld.avg	sector	0
lltex__t_sectors_pipe_lsu_mem_local_op_ld.max	sector	0
lltex__t_sectors_pipe_lsu_mem_local_op_ld.min	sector	0
lltex__t_sectors_pipe_lsu_mem_local_op_ld.sum	sector	0
sm__sass_inst_executed_op_local.avg	inst	0
sm__sass_inst_executed_op_local.max	inst	0
sm__sass_inst_executed_op_local.min	inst	0
sm__sass_inst_executed_op_local.sum	inst	0
smsp__sass_branch_targets_threads_divergent.avg		1.88
smsp__sass_branch_targets_threads_divergent.max		8
smsp__sass_branch_targets_threads_divergent.min		0
smsp__sass_branch_targets_threads_divergent.sum		300

Выводы

В рамках лабораторной работы для решения системы линейных алгебраических уравнений (СЛАУ) применялся графический процессор (GPU), реализующий метод Гаусса. Для удобства работы с матрицей, представляющей собой двумерную структуру, была задействована двумерная сетка потоков, что позволило более эффективно распределить вычисления.

Проведённые тесты продемонстрировали высокую эффективность распараллеливания метода Гаусса на GPU, что позволило значительно сократить время выполнения алгоритма по сравнению с последовательной реализацией.